LOW LATENCY AND EFFICIENT VOICE ACTIVATOR FOR EDGE DEVICES

[Author Name(s)]
Department of Electronics and Communication Engineering, Malla Reddy University, Hyderabad, India
[Institutional e-mail address]

Abstract— Voice-based interaction provides a convenient hands-free interface, but cloud-dependent speech systems can introduce network delay, privacy concerns, and dependence on continuous connectivity. This paper presents the design of a low-latency voice activator for edge devices using an ESP32-S3-class microcontroller and a compact audio-processing pipeline. A digital microphone captures speech, which is framed and prepared for feature extraction. Mel-frequency cepstral coefficients (MFCCs) or log-Mel features are supplied to a lightweight keyword-spotting model for local inference. A command-decision stage validates the model output and triggers a local response through an OLED, LED, buzzer, or external control interface. Since recognition is intended to run on the device, the design avoids sending raw audio to a remote server during normal operation. The paper describes the architecture, signal-processing stages, model deployment, hardware and software integration, and an evaluation plan. Actual latency, accuracy, memory use, and power measurements must be recorded on the final assembled hardware; no measured performance values are claimed in this draft.

Index Terms— Edge artificial intelligence, ESP32-S3, keyword spotting, low-latency processing, MFCC, TinyML, voice activation.

I. INTRODUCTION

Voice activation enables users to control electronic systems through short spoken commands. It is useful in assistive devices, smart appliances, robotics, and industrial interfaces where physical interaction may be inconvenient. Conventional speech-recognition services often transmit audio to cloud servers. Although cloud processing can support large-vocabulary recognition, the communication path introduces network-dependent latency and may expose private audio. An internet connection is also required for continuous operation.

Edge intelligence provides an alternative by performing selected inference tasks directly on an embedded device. Keyword spotting identifies a small set of trigger words or commands rather than transcribing unrestricted speech. A compact neural network can therefore operate within the memory, computation, and energy limits of a microcontroller. The proposed design captures audio through an I2S microphone, prepares the signal, performs local inference, and activates a mapped output.

A. Objectives

The objectives are to acquire speech through a digital microphone; prepare audio using framing, normalization, and compact spectral features; detect a defined wake word or command with a lightweight model; perform inference locally; provide a clear output or control action; and evaluate response time, recognition accuracy, memory consumption, and power after implementation.

B. Scope

The scope is limited to a predefined vocabulary and embedded keyword recognition. It does not claim unrestricted speech-to-text capability, speaker identification, or cloud-scale language understanding. Vocabulary, sampling rate, frame duration, and output actions must be finalized during implementation.

II. RELATED WORK

Keyword spotting has been studied as a compact speech-recognition problem. Warden introduced the Speech Commands dataset for reproducible evaluation of limited-vocabulary speech recognition [1]. MFCCs are widely used in speech recognition; Davis and Mermelstein compared parametric representations for isolated-word recognition [2]. TinyML research addresses deployment of machine-learning models on resource-constrained devices [3]. TensorFlow Lite for Microcontrollers provides a runtime intended for low-memory embedded targets [4].

III. PROPOSED SYSTEM ARCHITECTURE

The architecture consists of audio acquisition, signal pre-processing, feature extraction, a keyword-spotting model, command validation, and an output interface. The stages form a local processing pipeline. Raw audio is not required to leave the edge device for the intended wake-word and command-recognition operation.

Audio acquisition → pre-processing → MFCC/log-Mel features → TinyML keyword model → command decision → output interface.

Fig. 1. Functional architecture of the proposed edge voice activator.

A. Audio Acquisition

A digital MEMS microphone such as the INMP441 can be interfaced through the Inter-IC Sound (I2S) peripheral. I2S carries serial audio data together with bit-clock and word-select signals. The ESP32-S3 receives samples into a buffer. The selected microphone, pin assignment, sample width, and sampling frequency must be verified against the actual module and firmware configuration.

B. Pre-processing

Incoming audio is divided into short, overlapping frames. A window function reduces discontinuities at frame boundaries. Optional filtering can attenuate unwanted noise, while normalization can reduce sensitivity to changes in recording amplitude. Processing parameters must remain consistent with the training pipeline; otherwise, the deployed model may receive features that differ from those used during training.

C. Feature Extraction

MFCC or log-Mel filter-bank features convert the time-domain waveform into a compact representation of short-time spectral content. A typical MFCC pipeline applies a short-time Fourier transform, a mel-spaced filter bank, logarithmic compression, and a discrete cosine transform. The number of frames and coefficients determines feature tensor size and affects memory use and inference time.

D. Keyword-Spotting Model

A lightweight neural network maps the feature tensor to a small set of output classes, such as a wake word, supported commands, and a background or unknown class. A compact convolutional architecture is a candidate because local time-frequency patterns can be learned with relatively low computational cost. Model architecture, quantization, and tensor dimensions must be selected according to available flash and RAM.

E. Command Decision and Output

The model output is passed to a decision stage. A confidence threshold and optional consecutive-frame confirmation can reduce false activations. When a command is accepted, firmware selects the corresponding action, updates the OLED, switches an LED or buzzer, or operates an external interface. Invalid or uncertain predictions should leave outputs in a defined safe state.

IV. HARDWARE REQUIREMENTS

The proposed prototype may use an ESP32-S3 development board, an I2S digital microphone such as INMP441, an OLED display, an LED, a buzzer, a push button, and a regulated power supply appropriate to the selected board. A MAX98357A amplifier and speaker may be included when audible feedback is required. A relay or driver stage is optional and must be selected according to load voltage and current.

The exact board revision and module pinout must be checked before wiring. The microphone requires correct I2S clock, word-select, and data connections, and all modules must share a suitable ground. Power rails must not exceed component ratings. A relay must not be driven directly from a microcontroller GPIO without an appropriate driver and protection arrangement.

V. SOFTWARE DESIGN AND IMPLEMENTATION

A. Development Environment

Firmware can be developed using the Arduino IDE or ESP-IDF, depending on the required control over audio drivers, memory allocation, and task scheduling. Model training and dataset preparation can be performed on a computer using Python and a machine-learning framework. The trained model is converted to a format supported by the embedded inference runtime, such as TensorFlow Lite for Microcontrollers.

B. Audio Buffering

The audio driver collects samples into fixed-size buffers. Double buffering or a ring buffer can separate acquisition from feature computation so that microphone sampling is not blocked while the model runs. Buffer length should match the frame configuration. Buffer-overrun and dropped-sample counters should be included during development.

C. Feature Pipeline

For every analysis window, firmware applies the same pre-processing and feature extraction used during model training. If MFCC features are used, the implementation must match the training sample rate, frame length, frame step,

window function, mel filter-bank configuration, coefficient count, and scaling. These parameters should be stored as configuration constants and documented with the model version.

D. Model Deployment

The trained model is converted to an embedded representation and linked with the firmware. Integer quantization can reduce model size and improve execution efficiency where supported. Quantization must be validated against a floating-point baseline because accuracy may change after conversion. Firmware should check model initialization and tensor-arena allocation before entering the recognition loop.

E. Command Handling

The output class with the highest score is not automatically treated as a valid command. The command handler compares the score with a configured threshold and may require the same class in consecutive windows. A short refractory interval can prevent repeated triggering from one utterance. Unknown speech and low-confidence predictions should be ignored.

VI. ALGORITHM

1) Initialize the I2S microphone, buffers, display, indicators, and inference runtime. 2) Capture a fixed-length audio window. 3) Apply pre-processing and divide the signal into analysis frames. 4) Extract MFCC or log-Mel features using the training configuration. 5) Run the lightweight keyword-spotting model on the ESP32-S3. 6) Evaluate the output class and confidence threshold. 7) If a valid command is detected, execute the mapped action and provide feedback. 8) Otherwise, continue listening and repeat the process.

VII. COMMUNICATION AND SYNCHRONIZATION

The microphone communicates with the controller through I2S. The OLED typically uses I2C, while LEDs and control signals use general-purpose input/output pins. I2S transports sampled audio, I2C transfers display commands and data, and GPIO provides digital control. If a speaker is included through a digital audio amplifier, its interface and sample format must be configured separately.

Audio acquisition and inference should be coordinated so that feature extraction does not cause gaps in sampling. A producer-consumer arrangement is one option: the acquisition task places completed buffers into a queue, and the processing task consumes them. Queue depth, task priority, and buffer ownership must prevent overwriting data that is still being processed.

VIII. MODEL TRAINING AND OPTIMIZATION

A. Dataset Preparation

A keyword-spotting model requires labeled recordings for every supported command and for background or unknown speech. Recordings should include different speakers, speaking rates, distances, and acoustic environments. Training, validation, and test partitions should be separated by recording or speaker where possible to reduce data leakage. Background noise and silence examples are important because the device spends much of its time listening when no command is present.

B. Data Augmentation

Training examples may be augmented with controlled background noise, small time shifts, and amplitude changes. Augmentation should reflect plausible operating conditions rather than introduce unrealistic distortions. The same audio normalization and feature extraction used during training must be reproduced on the embedded target.

C. Model Compression

A small model is preferred to reduce flash storage, RAM usage, and inference time. Candidate optimization methods include reducing filters or layers, integer quantization, and selecting a compact input representation. The effect of each optimization should be measured independently. Compression is successful only when the resulting model continues to meet recognition requirements.

D. False Activation Control

False activation can occur when background speech or environmental sounds resemble a target keyword. A background class, confidence threshold, temporal smoothing, and a short confirmation rule can reduce unintended triggering. Stricter thresholds may reduce false positives but can also reject genuine commands. Threshold selection should therefore be based on validation data and the intended environment.

IX. SYSTEM INTEGRATION

The hardware and software are integrated in stages. First, the microphone is tested independently by recording or displaying sample amplitudes. Second, feature extraction is checked using known audio samples. Third, the model is tested with stored feature vectors before live microphone inference is enabled. Finally, output devices are connected and each recognized command is checked against the expected action.

A modular implementation simplifies debugging. Audio acquisition, feature extraction, inference, decision logic, and output control should be separate functions or tasks. Serial diagnostics can report buffer status, model initialization, predicted class, and confidence during development. Diagnostic printing should be disabled or reduced during final timing measurements because serial output can affect response time.

A. Suggested Pin Mapping

A suggested wiring arrangement assigns I2S clock to GPIO4, word-select to GPIO2, and data input to GPIO3; OLED SDA and SCL to GPIO8 and GPIO9; LED to GPIO5; buzzer to GPIO6; and an optional relay-control signal to GPIO7. This mapping is only a starting point. It must be checked against the exact ESP32-S3 board, boot-strapping pins, and peripheral conflicts before use.

B. Reliability and Safety

The device should start in a known inactive state. Invalid model outputs must not trigger an external load. If a relay is used, the driver stage, flyback protection where applicable, isolation, load rating, and enclosure must be considered. Firmware should handle microphone initialization failure, model allocation failure, and unexpected buffer conditions without producing uncontrolled outputs.

X. PERFORMANCE EVALUATION

The system should be evaluated using repeatable tests on the final hardware. Recommended metrics are end-to-end activation latency, keyword accuracy, false-accept rate, false-reject rate, model flash size, peak RAM or tensor-arena use, and average current or power. Test conditions, utterance count, speaker count, noise level, and distance should be reported.

A. Latency Measurement

Latency should be measured from a clearly defined start event to a clearly defined output event. For example, the start may be the end of the spoken keyword, and the end may be the LED transition or command output. Report mean, median, and a high percentile over repeated trials. Separate capture-window delay, feature computation time, model inference time, and output delay where possible.

B. Recognition Measurement

A confusion matrix should be constructed for target commands and the background class. Accuracy alone may hide false activations when the background class dominates. Report per-class precision and recall, false accepts per hour of background audio, and false rejects for valid commands. Testing should include speakers and environments not used during training.

XI. RESULTS AND DISCUSSION

The project information available for this draft identifies the intended ESP32-S3-based architecture, I2S microphone acquisition, MFCC or log-Mel feature processing, lightweight keyword recognition, and local output control. However, no verified experimental readings, final model file, or completed test log were supplied. Therefore, numerical latency, accuracy, memory, and power results are intentionally not fabricated.

After the prototype is tested, measured values should be inserted in Table I. Use the same firmware build and model version for all measurements. For latency, document whether the value includes the complete audio-window collection period. For accuracy, state the number of positive and background samples and explain how the test set was separated from training data.

Metric	Measurement to be entered
End-to-end activation latency	[Measured value] ms
Mean model inference time	[Measured value] ms
Keyword recognition accuracy	[Measured value] %
False accepts	[Count or rate]
False rejects	[Count or rate]
Model flash footprint	[Measured value] kB
Peak RAM / tensor arena	[Measured value] kB
Average operating current	[Measured value] mA

TABLE I. EXPERIMENTAL RESULTS TEMPLATE — COMPLETE AFTER HARDWARE TESTING.

A. Expected Design Benefits

Local inference removes the network round trip from the normal recognition path and permits operation without continuous internet connectivity. It also limits routine transmission of microphone audio to external services. The compact vocabulary and embedded model reduce computational requirements compared with unrestricted speech recognition. These are design-level benefits; actual latency and energy advantages must be confirmed by measurement against a defined baseline.

B. Design Trade-Offs

A smaller model can reduce memory use and inference time but may lose recognition accuracy. Increasing the audio window can improve the context available to the classifier but adds waiting time. More aggressive filtering may suppress noise but can also remove useful speech information. The final configuration should be selected using measured results from the intended operating environment.

XII. LIMITATIONS

The proposed system is a limited-vocabulary voice activator, not a general-purpose conversational assistant. Performance may be affected by microphone placement, reverberation, background noise, accent variation, and similarity between command words. The selected model and runtime must fit available memory and supported operators of the final board. A fixed confidence threshold may not be optimal across all environments.

The exact ESP32-S3 board revision, final microphone module, trained model parameters, command vocabulary, sampling configuration, and measured prototype results remain to be confirmed. The pin mapping and optional output modules described here are implementation suggestions and must be validated on the actual hardware.

XIII. CONCLUSION

This paper presented the design of a low-latency voice activator intended for edge devices. The proposed architecture uses a digital microphone to acquire speech, an embedded controller to prepare audio and extract compact features, and a lightweight keyword-spotting model to identify a predefined set of commands locally. A command-decision stage then activates a display, indicator, buzzer, or optional external interface. The approach is intended to reduce dependence on cloud connectivity and avoid routine transmission of audio for the supported commands.

The paper described implementation requirements including consistent training and deployment pre-processing, audio buffering, model conversion, command validation, and hardware integration. A reproducible evaluation procedure was defined for latency, recognition performance, memory footprint, and power consumption. Since verified prototype measurements were not provided, quantitative performance claims have been left as fields to be completed after testing.

XIV. FUTURE SCOPE

Future work can extend the command vocabulary while maintaining a small memory footprint. Noise-robust training, speaker diversity, adaptive thresholding, and model quantization may improve practical performance. A low-power listening mode and optimized task scheduling may reduce energy consumption. Additional outputs, such as safe appliance control or robot commands, can be integrated through suitable driver circuits. A future version may compare different feature representations and model architectures using the same dataset and hardware platform.

ACKNOWLEDGMENT

The authors may acknowledge the support of the Department of Electronics and Communication Engineering, Malla Reddy University, Hyderabad, and any individuals or laboratories that contributed to the prototype. Revise this section to reflect actual support received.

REFERENCES

[1] P. Warden, “Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition,” arXiv:1804.03209, 2018.

[2] S. B. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” IEEE Trans. Acoust., Speech, Signal Process., vol. 28, no. 4, pp. 357–366, Aug. 1980.

[3] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. Sebastopol, CA, USA: O’Reilly Media, 2019.

[4] A. Lin et al., “MCUNet: Tiny Deep Learning on IoT Devices,” in Proc. Advances in Neural Information Processing Systems, 2020.

[5] TensorFlow, “TensorFlow Lite for Microcontrollers,” documentation. [Online]. Available: https://www.tensorflow.org/lite/microcontrollers

[6] Espressif Systems, “ESP32-S3 Series Datasheet,” technical documentation. [Online]. Available: https://www.espressif.com/

APPENDIX — IMPLEMENTATION CHECKLIST

Confirm the exact ESP32-S3 board and GPIO availability; verify microphone supply, I2S pinout, sample format, and sampling rate; record the command vocabulary and collect labeled speech/background samples; document feature parameters; train and validate the model with a reserved test partition; convert and quantize if required; verify audio buffers, model initialization, thresholds, and output behavior; measure latency, recognition metrics, memory, and current on the assembled prototype.

AUTHOR INFORMATION

[Insert the full name(s) of the student author(s), guide/supervisor name, department, university, and institutional e-mail address. These details were not supplied and remain editable.]

Author biographies and photographs may be added if required by the target publication. Sample author names and biographies from the original IEEE template have not been retained.